Gmail

Srinivasan Kannan <ka.shreenivaasan@gmail.com>

ProofNet from academic papers and Knowledge graphs

6 messages

Srinivasan Kannan <ka.shrinivaasan@gmail.com>
To: Srinivasan Kannan <ka.shreenivaasan@gmail.com>

Tue, Aug 11, 2026 at 11:21 AM

Yes. I can create a **ProofNet / proof network from the theorems and proofs in an academic publication**, including dependencies between definitions, lemmas, propositions, theorems, and corollaries.

For example, I can transform a paper into a structure like:

```
Definitions
    │
    ├── Definition 1 ──────┐
    │                      ↓
    ├── Lemma 2 ──────────→ Proposition 3
    │                      │
    └── Lemma 4 ───────────┘
                           ↓
                      Theorem 5
                      /       \
                     ↓         ↓
              Corollary 6   Theorem 7
```

I can produce several levels of proofnet:

1. **Theorem dependency graph** — which theorems depend on which earlier results.
2. **Proof-step proofnet** — individual logical steps within each proof.
3. **Definition → Lemma → Theorem network** — tracks foundational concepts.
4. **Cross-theorem knowledge graph** — connects concepts, assumptions, equations and results.
5. **Formal proofnet** — where possible, express the argument in a formal logical representation such as propositions, premises, inference rules and conclusions.
6. **Machine-readable output** — JSON, GraphML, RDF/OWL, Mermaid, DOT/Graphviz, or HTML visualization.

What I need

Upload the academic paper (PDF) or give me its URL/DOI/title.

I can then extract something like:

ID	Type	Statement	Depends on
D1	Definition	...	—

ID	Type	Statement	Depends on
L1	Lemma	...	D1
L2	Lemma	...	D1, L1
P1	Proposition	...	L2
T1	Theorem	...	P1, L2
C1	Corollary	...	T1

and construct the complete proofnet.

If you give me a **mathematics/CS theoretical paper**, I can also attempt to distinguish **explicitly cited dependencies** from **dependencies that are logically required by the proof**, which makes the resulting network substantially more useful for research.

NeuronRain Documentation and Licensing: http://neuronrain-documentation.readthedocs.io/en/latest/
Personal website(research): https://acadpdrafts.readthedocs.io/en/latest/

Srinivasan Kannan <ka.shrinivaasan@gmail.com>
To: Srinivasan Kannan <ka.shreenivaasan@gmail.com>
Tue, Aug 11, 2026 at 11:22 AM

Yes. The **Cook–Levin theorem** is an excellent example because its proof naturally forms a multi-level proofnet.

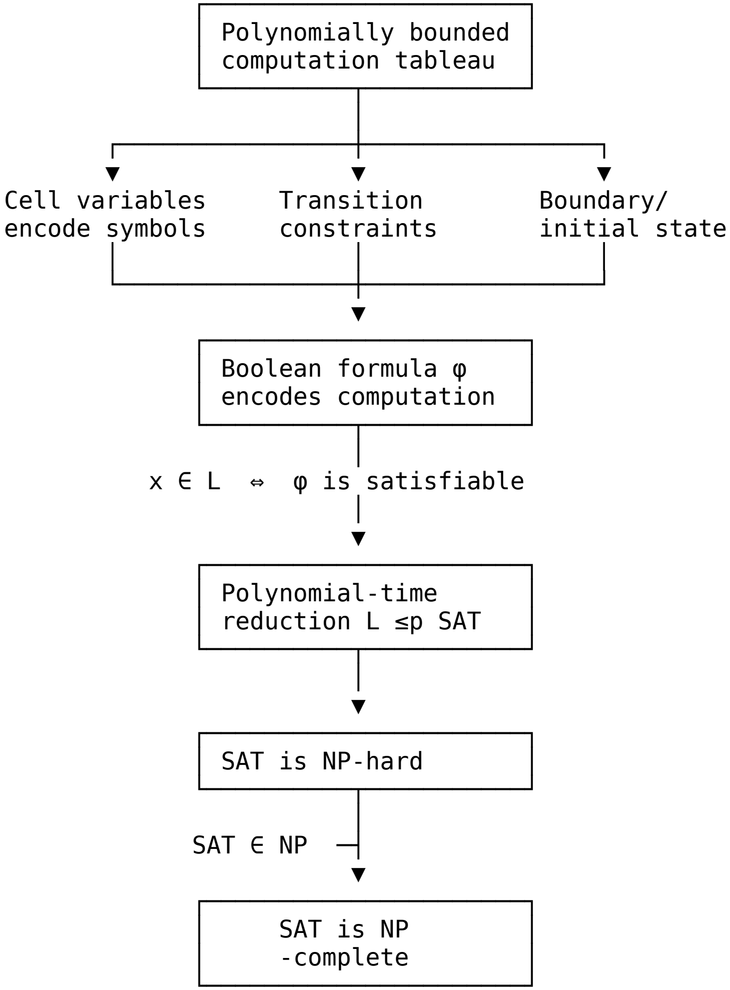
Cook–Levin ProofNet

The central theorem is:

> **Theorem (Cook–Levin):** Boolean satisfiability (SAT) is NP-complete.

A proofnet can be represented as:

```
┌──────────────────────┐
│ Polynomial-time      │
│ verifier definition  │
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│ NP machine M         │
│ accepting input x    │
└──────────┬───────────┘
           │
 accepting computation
           │
           ▼
```



More formal proofnet

We can turn the proof into nodes and directed dependency edges:

Node	Mathematical content	Depends on
D1	Definition of NP	—
D2	Polynomial-time verifier	D1
D3	Nondeterministic polynomial-time TM	D1
L1	Verifier ↔ nondeterministic computation	D2, D3
D4	Computation tableau	D3
L2	Tableau has polynomial size	D3

Node	Mathematical content	Depends on
D5	Variables representing tableau cells	D4
L3	Local transition constraint	D5
L4	Initial-configuration constraint	D5
L5	Accepting-configuration constraint	D5
L6	Local constraints can be encoded as Boolean clauses	L3
L7	Combined formula has polynomial size	L2, L6
T1	$x \in L \Rightarrow \varphi x$ is satisfiable	L3–L5
T2	φx satisfiable $\Rightarrow x \in L$	L3–L5
T3	$x \in L \Leftrightarrow \varphi x \in SAT$	T1, T2
L8	Reduction computable in polynomial time	L7
T4	Every L ∈ NP reduces to SAT	T3, L8
T5	SAT is NP-hard	T4
L9	SAT ∈ NP	verifier for satisfying assignment
T6	**SAT is NP-complete**	**T5, L9**

So the core proofnet is:

```
D1 ──→ D2 ──→ L1 ──────────────┐
 │                             │
 └──→ D3 ──→ D4 ──→ D5 ──→ L3 ─┤
              │          ├──→ L4 ─┤
              │          └──→ L5 ─┤
              ↓                   ↓
              L2 ────────────→ L7
                                 │
                                 ↓
                                L8
                                 │
                                 ↓
                                T3
                              /    \
                            T1      T2
                              \    /
                               ↓
                               T4
                               │
                               ↓
                               T5
                               │
                   ┌───────────┘
                   ↓
        SAT ∈ NP ──────────→ T6
                             │
```

```
                 ▼
          SAT is NP-complete
```

The particularly interesting part

A **ProofNet is more informative than simply extracting the theorem statements**.

For Cook–Levin, we can represent the crucial semantic equivalence as:

$$x\in L \quad\Longleftrightarrow\quad M\text{ accepts }x \quad\Longleftrightarrow\quad \text{there exists an accepting computation tableau}$$
$$\Longleftrightarrow \text{there exists a satisfying assignment} \quad\Longleftrightarrow\quad \varphi_x\in SAT.$$

Thus the proofnet connects **complexity-theoretic concepts** → **computation** → **combinatorial representation** → **Boolean logic** → **reduction** → **NP-completeness**.

If you provide a paper, I can build this same type of , with , and export it as **Graphviz/Mermaid + JSON + an interactive HTML graph**.

NeuronRain Documentation and Licensing: http://neuronrain-documentation.readthedocs.io/en/latest/
Personal website(research): https://acadpdrafts.readthedocs.io/en/latest/

[Quoted text hidden]

Srinivasan Kannan <ka.shrinivaasan@gmail.com> Tue, Aug 11, 2026 at 11:23 AM
To: Srinivasan Kannan <ka.shreenivaasan@gmail.com>

Yes. The **Erdős distance problem/theorem** is a very good example because its modern proof network has several distinct layers: incidence geometry, polynomial partitioning, combinatorial energy, and the final distance-counting argument.

There are actually several results called the **Erdős distance theorem**. The classical statement is:

$$\boxed{\text{Every set of }n\text{ points in }\mathbb R^2 \text{ determines at least } c\frac{n}{\sqrt{\log n}} \text{ distinct distances.}}$$

A ProofNet for this result can be represented as follows.

1. High-level ProofNet

```
flowchart TD

D1["Definition: distinct distances Δ(P)"]
D2["Definition: distance energy"]
D3["Definition: incidences"]
D4["Definition: isosceles triangles"]

L1["Lemma: distance multiplicities"]
L2["Lemma: Energy–distance relation"]
L3["Lemma: Isosceles triangles ↔<br/>distance multiplicities"]
```

```
P1["Proposition: Many repeated distances<br/>imply many isosceles triangles"]

L4["Incidence bound"]
L5["Szemerédi–Trotter theorem"]
L6["Incidence corollary for circles"]

P2["Proposition: Number of isosceles<br/>triangles is bounded"]

P3["Proposition: Distance energy<br/>has an upper bound"]

L7["Cauchy–Schwarz inequality"]

P4["Proposition: Lower bound on<br/>number of distinct distances"]

T1["Erdős distance theorem"]

D1 --> L1
D2 --> L2
D3 --> L4
D4 --> L3

L1 --> L2
L3 --> P1

L5 --> L6
L6 --> P2
L4 --> P2

P1 --> P3
P2 --> P3

L7 --> P4
P3 --> P4

P4 --> T1
```

The important structure is:

$$ \text{incidence theorem} \rightarrow \text{isosceles-triangle bound} \rightarrow \text{energy bound} \rightarrow \text{distance bound}. $$

---

2. A more mathematical ProofNet

Let

$$ P=\{p_1,\ldots,p_n\}\subset\mathbb R^2 $$

and let

$$ D(P)=\{\,|p_i-p_j|:i<j\,\}. $$

The target is

$$ |D(P)|\gtrsim\frac{n}{\sqrt{\log n}}. $$

We can construct the network as:

```
                 P ⊂ R²
                   │
                   ▼
           Pairwise distances
                   │
       ┌───────────┴───────────┐
       ▼                       ▼
Distance multiplicities     Isosceles
     m(r)                    triangles
       │                       │
       └───────────┬───────────┘
                   ▼
         Distance energy E(P)
                   │
                   ▼
     ┌─────────────────────────┐
     │ Incidence estimate      │
     │                         │
     │ Szemerédi–Trotter       │
     └─────────────────────────┘
                  │
                  ▼
     Bound on E(P)
                  │
                  ▼
       Cauchy–Schwarz
                  │
                  ▼
    |D(P)| ≥ c n / √log n
                  │
                  ▼
      Erdős distance theorem
```

3. Node-level representation

ID	Type	Result	Dependencies
D1	Definition	$D(P)$, distinct distances	—
D2	Definition	distance multiplicity $m(r)$	D1
D3	Definition	distance energy $E(P)$	D2
D4	Definition	isosceles-triangle count	D2
L1	Lemma	$\sum_r m(r)=\binom n2$	D2

ID	Type	Result	Dependencies
L2	Lemma	energy expressed through multiplicities	D3
L3	Lemma	repeated distances generate isosceles configurations	D2,D4
L4	Theorem	Szemerédi–Trotter incidence bound	D3
L5	Proposition	point-circle incidence bound	L4
L6	Proposition	bound on relevant isosceles configurations	L5
P1	Proposition	upper bound on distance energy	L3,L6
L7	Lemma	Cauchy–Schwarz energy inequality	L1,D3
P2	Proposition	lower bound on □	D(P)
T1	**Theorem**	**Erdős distinct-distance lower bound**	**P2**

---

4. The key logical chain

The ProofNet becomes especially interesting when we represent the mathematics rather than just the names of the results.

The first transformation is:

$$ \binom n2 = \sum_{r\in D(P)}m(r). $$

Then Cauchy–Schwarz gives a relationship of the form

$$ \left(\sum_r m(r)\right)^2 \leq |D(P)|\sum_r m(r)^2. $$

Therefore,

$$ |D(P)| \geq \frac{\binom n2^2} {\sum_r m(r)^2}. $$

So the entire problem becomes:

$$ \boxed{\text{upper-bound }\sum_r m(r)^2.} $$

And that quantity is precisely where the geometric/incidence part of the proof enters.

Thus the ProofNet has a very useful **problem transformation node**:

```
     Erdős distance problem
               │
               ▼
  Count distance multiplicities
               │
               ▼
     Distance energy
               │
               ▼
  ┌──────────────────┐
  │ Need upper bound on │
```

```
│ Σ m(r)²              │
└──────────┬───────────┘
           │
           ▼
 Incidence geometry
           │
           ▼
Bound repeated-distance
    configurations
           │
           ▼
     Energy bound
           │
           ▼
 Cauchy–Schwarz
           │
           ▼
Distinct-distance bound
```

An important historical distinction

For the **original Erdős–Moser/Erdős distance problem**, the best-known bound has evolved considerably. The classical \(n/\sqrt{\log n}\) result is associated with the Erdős distance theorem, while later work by , , , and others gives stronger results in different formulations and dimensions.

So if the goal is a **publication-faithful ProofNet**, I would use the exact paper you specify and extract **every numbered Definition, Lemma, Proposition, Corollary and Theorem**, rather than mixing the classical and modern proofs.

If you upload the **Erdős distance theorem paper/PDF**, I can and identify **which exact lemma feeds each theorem**, rather than just giving this conceptual network.

NeuronRain Documentation and Licensing: http://neuronrain-documentation.readthedocs.io/en/latest/
Personal website(research): https://acadpdrafts.readthedocs.io/en/latest/

[Quoted text hidden]

Srinivasan Kannan <ka.shrinivaasan@gmail.com>
To: Srinivasan Kannan <ka.shreenivaasan@gmail.com>
Tue, Aug 11, 2026 at 11:24 AM

For the **Pythagorean theorem**, we can make the ProofNet much more compact and precise because there are many possible proofs. Here is a canonical **Euclidean similarity proof**.

Pythagorean Theorem ProofNet

Let \(ABC\) be a right triangle with

\[\angle C=90^\circ,\qquad AC=b,\quad BC=a,\quad AB=c. \]

The target is

\[a^2+b^2=c^2. \]

```
D1  Right triangle ABC
        │
        ▼
D2  Altitude CD ⊥ AB
        │
        ▼
L1  ∠ACD = ∠ABC
        │
        ▼
L2  ∠BCD = ∠BAC
        │
        ├──────────────────┐
        ▼                  ▼
T1  △ACD ~ △ABC      T2  △BCD ~ △ABC
        │                  │
        ▼                  ▼
P1  AC² = AD·AB      P2  BC² = BD·AB
        │                  │
        └────────┬─────────┘
                 ▼
L3  AC² + BC²
    = AB(AD+BD)
                 │
                 ▼
L4  AD + BD = AB
                 │
                 ▼
T3  AC² + BC² = AB²
                 │
                 ▼
       PYTHAGOREAN
        THEOREM
```

Node/edge representation

Node	Type	Statement	Dependencies
D1	Definition	\(ABC\) is right-angled at \(C\)	—
D2	Construction	Draw altitude \(CD\) to hypotenuse \(AB\)	D1
L1	Lemma	\(\angle ACD=\angle ABC\)	D1,D2
L2	Lemma	\(\angle BCD=\angle BAC\)	D1,D2
T1	Theorem	\(\triangle ACD\sim\triangle ABC\)	L1
T2	Theorem	\(\triangle BCD\sim\triangle ABC\)	L2
P1	Proposition	\(AC^2=AD\cdot AB\)	T1

Node	Type	Statement	Dependencies
P2	Proposition	\(BC^2=BD\cdot AB\)	T2
L3	Lemma	\(AC^2+BC^2=AB(AD+BD)\)	P1,P2
L4	Lemma	\(AD+BD=AB\)	D2
T3	**Theorem**	**\(AC^2+BC^2=AB^2\)**	**L3,L4**

The actual logical kernel

The interesting part of the ProofNet is that the famous theorem is actually the terminal node of a dependency chain:

\[\boxed{ \text{right angle} \rightarrow \text{altitude} \rightarrow \text{angle equalities} \rightarrow \text{similar triangles} \rightarrow \text{geometric-mean relations} \rightarrow \text{segment addition} \rightarrow a^2+b^2=c^2 } \]

This is exactly the type of structure that can be extracted from an academic publication.

For a **real paper**, rather than a textbook proof, I can preserve the paper's exact theorem/lemma numbering and produce:

```
Paper
 ├── Definition 1
 ├── Lemma 2
 │    ├── Lemma 4
 │    └── Proposition 5
 ├── Theorem 6
 │    └── Corollary 7
 └── Theorem 8
      ├── Lemma 4
      ├── Proposition 5
      └── Theorem 6
```

with every edge representing an identified **proof dependency**.

NeuronRain Documentation and Licensing: http://neuronrain-documentation.readthedocs.io/en/latest/
Personal website(research): https://acadpdrafts.readthedocs.io/en/latest/

[Quoted text hidden]

Srinivasan Kannan <ka.shrinivaasan@gmail.com> Tue, Aug 11, 2026 at 11:25 AM
To: Srinivasan Kannan <ka.shreenivaasan@gmail.com>

Yes. For the **Pythagorean theorem**, the ProofNet can be converted into a **knowledge graph** by separating mathematical entities from propositions and relationships.

Pythagorean Theorem Knowledge Graph

```
graph TD

A["Right Triangle ABC"]
B["∠C = 90°"]
C["AC = b"]
D["BC = a"]
E["AB = c"]

F["Altitude CD"]
G["CD ⊥ AB"]
H["Point D on AB"]

I["△ACD"]
J["△BCD"]
K["△ABC"]

L["∠ACD = ∠ABC"]
M["∠BCD = ∠BAC"]

N["△ACD ~ △ABC"]
O["△BCD ~ △ABC"]

P["AC² = AD × AB"]
Q["BC² = BD × AB"]

R["AD + BD = AB"]

S["AC² + BC² = AB(AD + BD)"]
T["AC² + BC² = AB²"]

U["Pythagorean Theorem"]

A --> B
A --> C
A --> D
A --> E

A --> F
F --> G
F --> H

F --> L
F --> M

L --> N
M --> O

N --> P
O --> Q

P --> S
```

```
Q --> S
H --> R

R --> T
S --> T

T --> U
```

Knowledge-graph triples

The same graph can be represented as RDF-style triples:

```
(ABC, type, RightTriangle)
(ABC, hasAngle, C)
(C, measure, 90°)

(ABC, hasSide, AC)
(AC, length, b)

(ABC, hasSide, BC)
(BC, length, a)

(ABC, hasSide, AB)
(AB, length, c)

(CD, isAltitudeOf, ABC)
(CD, perpendicularTo, AB)
(D, liesOn, AB)

(ACD, similarTo, ABC)
(BCD, similarTo, ABC)

(ACD, implies, AC² = AD·AB)
(BCD, implies, BC² = BD·AB)

(AD + BD, equals, AB)

(AC² = AD·AB, combinedWith, BC² = BD·AB)
(AC² + BC², equals, AB(AD+BD))

(AD+BD, equals, AB)
(AB(AD+BD), equals, AB²)

(AC² + BC², equals, AB²)

(AC² + BC² = AB², instanceOf, PythagoreanTheorem)
```

The important difference from the ProofNet

A **ProofNet** primarily asks:

> **What result depends on what?**

A **knowledge graph** asks:

> **What mathematical objects, concepts, propositions and relationships are present, and how are they related?**

So we can combine them:

```
          ┌────────────────┐
          │ Right Triangle │
          └────────────────┘
                 │ has
                 ▼
          ┌────────────────┐
          │ Right Angle    │
          └────────────────┘
                 │ enables
                 ▼
          ┌────────────────┐
          │ Altitude CD    │
          └────────────────┘
                 │ creates
          ┌──────┴─────────┐
          ▼                ▼
   Triangle ACD       Triangle BCD
          │ similar to       │ similar to
          ▼                ▼
   Triangle ABC       Triangle ABC
          │                │
          └──────┬─────────┘
                 ▼
        Geometric relations
                 │
                 ▼
           AC² + BC² = AB²
                 │
                 ▼
        Pythagorean Theorem
```

This suggests a useful **Academic Knowledge Graph + ProofNet** architecture for the papers you mentioned earlier:

Paper → **Definitions** → **Concepts** → **Lemmas** → **Propositions** → **Theorems** → **Proof steps** → **Dependencies** → **Corollaries** → **Applications**

For a full academic paper, I can make each theorem/lemma/proposition a node while also creating nodes for the **mathematical concepts and objects appearing inside them**. That would give you a much richer graph than a simple citation/dependency network.

NeuronRain Documentation and Licensing: http://neuronrain-documentation.readthedocs.io/en/latest/
Personal website(research): https://acadpdrafts.readthedocs.io/en/latest/

[Quoted text hidden]

Srinivasan Kannan <ka.shrinivaasan@gmail.com>
To: Srinivasan Kannan <ka.shreenivaasan@gmail.com>
Tue, Aug 11, 2026 at 11:25 AM

Not equivalent in the strict sense. But there is a deep connection, and your Knowledge Graph + ProofNet idea can be viewed as moving toward some of the same territory as **Girard's Geometry of Interaction (GoI)**.

The key distinction

Your ProofNet / Knowledge Graph	Girard's Geometry of Interaction
Nodes represent concepts, definitions, lemmas, propositions, theorems	Objects represent components of proofs/programs
Edges represent dependencies/relations	Interaction represents computational execution
Primarily a **static structure**	Fundamentally a **dynamic/computational semantics**
"Theorem B depends on Lemma A"	"Proof A interacts with proof B"
Can represent mathematical knowledge	Models computation through proof interaction
Usually graph-theoretic	Based on linear logic, operators, and dynamics
Goal: organize/reconstruct mathematical reasoning	Goal: give a computational interpretation of proofs

Where they become very close

Consider our Pythagorean graph:

```
Right triangle
        │
        ▼
   Altitude
        │
        ▼
Similar triangles
        │
        ▼
Geometric relations
        │
        ▼
a² + b² = c²
```

A conventional Knowledge Graph says:

\[A\rightarrow B \]

meaning **A is related to / supports B**.

A ProofNet says:

$$A\rightarrow B$$

meaning **B depends on A in the proof**.

GoI asks a substantially different question:

$$\boxed{\text{What happens when proofs interact?}}$$

This is where **cut elimination** becomes central.

In the Curry–Howard perspective:

$$\text{proof} \leftrightarrow \text{program}$$

and GoI gives a way of interpreting the **execution of that proof/program** through interaction.

A possible bridge

There is, however, a very interesting architecture you could build:

```
MATHEMATICAL KNOWLEDGE GRAPH
           │
           ▼
    ┌──────────────┐
    │   ProofNet   │
    └──────────────┘
          │
dependency structure
          │
          ▼
   Formal proof graph
          │
     propositions
          │
          ▼
   Sequent calculus
          │
       cut rules
          │
          ▼
   Cut elimination
          │
          ▼
Geometry of Interaction
          │
          ▼
     Computation
```

That would be considerably closer to GoI.

The crucial difference

Suppose the graph contains:

$$ L_1\rightarrow P_1\rightarrow T. $$

Your ProofNet records the **dependency path**:

$$ L_1\leadsto T. $$

A GoI-style representation would need additional structure allowing the proof objects to **interact**, with composition and reduction corresponding to computation.

So:

$$ \boxed{\text{Knowledge Graph} \neq \text{ProofNet} \neq \text{GoI}} $$

but potentially:

$$ \boxed{ \text{Knowledge Graph} \rightarrow \text{ProofNet} \rightarrow \text{Formal Proof} \rightarrow \text{GoI} } $$

is a meaningful research direction.

In fact, if your ultimate objective is to turn **theorems from academic papers into an executable mathematical knowledge structure**, the interesting research question is not whether the ordinary Knowledge Graph is already GoI—it isn't—but **whether a ProofNet can be enriched with linear-logic/cut-elimination semantics so that theorem dependencies become an interaction/computation system**. That would be a much stronger construction than an ordinary academic knowledge graph.

NeuronRain Documentation and Licensing: http://neuronrain-documentation.readthedocs.io/en/latest/
Personal website(research): https://acadpdrafts.readthedocs.io/en/latest/

[Quoted text hidden]